

1.1. AVL fák

3.a, Adja meg az előadásról ismert **AVLremMin**(t, minp, d) rekurzív eljárás struktogramját, ami a $t \neq \emptyset$ AVL fából kivesszi a legkisebb kulcsú csúcsot, és minp -be teszi a címét! A d logikai típusú paraméterben azt kapjuk meg, hogy a művelet hatására csökkent-e a fa magassága. A kiegyensúlyozási szabályokat megvalósító eljárások közül elég a $(++,0)$ esetet kódolót részletezni, a szabálynak megfelelő ábrával együtt.

(AVLremMin(&t, &minp:Node* ; &d:ℤ))

$t \rightarrow \text{left} = \emptyset$		
$\text{minp} := t$	AVLremMin($t \rightarrow \text{left}, \text{minp}, d$)	
$t := \text{minp} \rightarrow \text{right}$		
$\text{minp} \rightarrow \text{right} := \emptyset$	d	
$d := \text{true}$	leftSubTreeShrunk(t, d)	SKIP

(leftSubTreeShrunk(&t:Node* ; &d:ℤ))

$t \rightarrow b = 1$	
$\text{/* } t \rightarrow b ++ \text{ */}$	$t \rightarrow b ++$
balance_PP(t, d)	$d := (t \rightarrow b = 0)$

(balance_PP(&t:Node* ; &d:ℤ))

$r := t \rightarrow \text{right}$		
$r \rightarrow b = -1$	$r \rightarrow b = 0$	$r \rightarrow b = 1$
balancePPm(t, r)	balancePP0(t, r)	balancePPp(t, r)
	$d := \text{false}$	

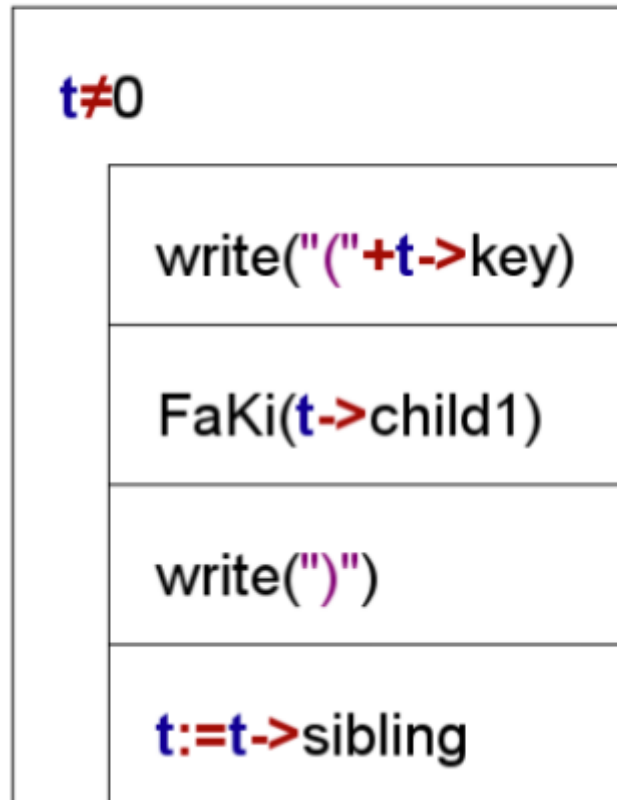
(balancePP0(&t, r : Node*))

$t \rightarrow \text{right} := r \rightarrow \text{left}$
$r \rightarrow \text{left} := t$
$t \rightarrow b := 1$
$r \rightarrow b := -1$
$t := r$

1.2. Általános fák

1.b, A t pointer egy binárisan láncolt általános fát azonosít. Írja meg a $\text{faKi}(t)$ eljárást, ami kiírja a fát szöveges (zárójeles) alakban, $\Theta(n)$ művelet-igénnyel és $O(h)$ tárigénnyel, ahol n a t fa mérete és h a magassága!

FaKi(t :Node*)



2. Gráfalgoritmusok

2. A $G[1..n]$ egy irányított gráf szomszédossági listás ábrázolása. Írja meg a **transzponál**(G, n, GT) eljárást, ami előállítja a $GT[1..n]$ gráfot, ami a $G[1..n]$ gráf transzponáltjának szomszédossági listás reprezentációja! A szomszédossági listák egyszerű láncolt listák (fejelem nélküli, nemciklikus, egyirányú listák). $MT(n, m) \in \Theta(n + m)$, ahol m a gráf éleinek száma.

Transzponált(**A**/1:Edge*[n];**AT**//1:Edge*[n])

i:=1 to n

AT[**i**]=0

i:= n downto 1

A[**i**]≠0

p:=**A**[**i**]

A[**i**]:=**A**[**i**]->next

u:=**p**->**v**

p->**v**:=**i**

p->next:=**AT**[**u**]

AT[**u**]:=p

3. $G[1..n]$ egy irányított gráf szomszédossági listás ábrázolása. A $G[i]$ listák egyszerű láncolt listák (fejelem nélküli, egyirányú, nemciklikus listák). Adja meg a listaelem típus leírását! Írja meg a **kibeFokok**(G, n, be, ki) eljárást, ami minden $u \in 1..n$ csúcsra a $be[u]$ -ban kiszámítja a csúcs bemeneti fokszámát, $ki[u]$ -ban pedig a kimeneti fokszámát! $MT(n, m) \in \Theta(n + m)$, ahol m a gráf éleinek száma.

ok[1]=1 es Kifok[1]=2 lenne

Befok_Kifok(A/1:Edge*[n];Befok/1:N[n];Kifok/1:N[n])

i:=1 to n

Befok[i]:=Kifok[i]:=0

i:=1 to n

p:=A[i]

p≠0

u:=p->v

Befok[u]++

Kifok[i]++

p:=p->next

$\Theta(1)$
 $\Theta(1)$

SP

2.1. Gráfrepresentációk

1. A $C[1..n, 1..n]$ bitmátrix egy irányított gráf szomszédossági mátrixos ábrázolása. Írja meg a **transzformál**(C, n, G) eljárást, ami előállítja a $G[1..n]$ gráfot, ami a $C[1..n, 1..n]$ mátrixszal reprezentált gráf szomszédossági listás reprezentációja! A szomszédossági listák egyszerű láncolt listák (fejelem nélküli, nemciklikus, egyirányú listák) legyenek! $MT(n) \in \Theta(n^2)$.

CsúcsMátrixbólÉlLista(A/1:bit[n,n];B/1:Edge*[n])

i:= 1 to n	
B[i]:=0	
j:= n downto 1	
T	A[i,j]=1
F	
p:=new Edge	Ø
p->v:=j	
p->next:=B[i]	
B[i]:=p	

2.2. Szélességi keresés

1.a, Mit számol ki a *Szélességi gráfkeresés*? **1.b,** Adja meg az algoritmus absztrakt struktogramját! **1.c,** A *Szélességi gráfkeresés* a gráf

$\text{BFS}(G : \mathcal{G} ; s : \mathcal{V})$	Műveletigény $MT(n,m)$	Műveletigény $mT(n,m)$
$\forall u \in G.V$		
$d(u) := \infty ; \pi(u) := \emptyset$	$\Theta(n)$	$\Theta(n)$
$[color(u) := white]$		
$d(s) := 0 ; [color(s) := grey]$	$\Theta(1)$	$\Theta(1)$
$Q : Queue ; Q.add(s)$	$\Theta(1)$	$\Theta(1)$
$\neg Q.isEmpty()$		
$u := Q.rem()$	$\Theta(n)$	$\Theta(1)$
$\forall v : (u, v) \in G.E$		csak a kezdőcsúcsot dolgozza fel a ciklus
$d(v) = \infty$	$\Theta(m)$	nullaszer lefutó ciklus is lehet, ha nincs éle a gráfnak az s csúcsból
$d(v) := d(u) + 1$		
$\pi(v) := u$		
$[color(v) := grey]$		
$Q.add(v)$		
$[color(u) := black]$	összesítve: $\Theta(n+m)$	összesítve: $\Theta(n)$

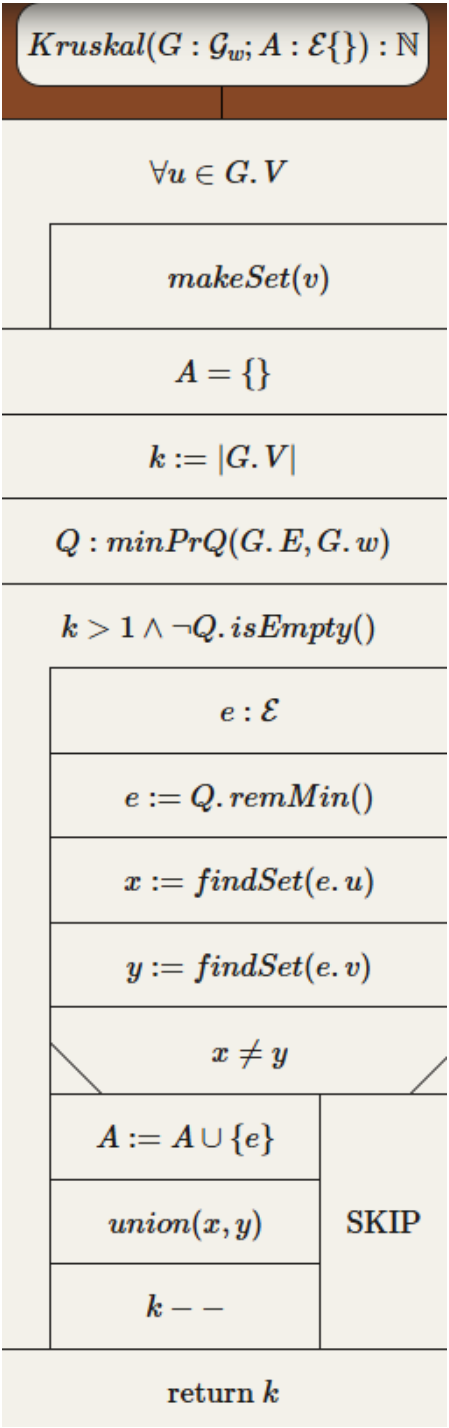
2.a, Rajzolja le a *Mélyégi gráfkeresés* absztrakt struktogramját! Mit tud a műveletigényéről? (Indokolja is az állítást!) **2.b,** Adja meg az éltípusok defi-

$\text{DFS}(G : \mathcal{G})$	$\text{DFvisit}(G : \mathcal{G} ; u : \mathcal{V} ; \&time : \mathbb{N})$
$\forall u \in G.V$	$d(u) := ++time ; color(u) := grey$
$color(u) := white$	$\forall v \in G.A(u)$
$time := 0$	$color(v) = white$
$\forall r \in G.V$	$\pi(v) := u \quad \quad color(v) = grey$
$color(r) = white$	$\text{DFvisit}(G, v, time) \quad \quad \text{backEdge}(u, v) \quad \quad \text{SKIP}$
$\pi(r) := \emptyset$	
$\text{DFvisit}(G, r, time)$	$f(u) := ++time ; color(u) := black$

Pontosán $3n+m+1$ lépést számolhatunk össze. Ebből a mélyégi keresésre $MT(n), mT(n) \in \Theta(n+m)$.

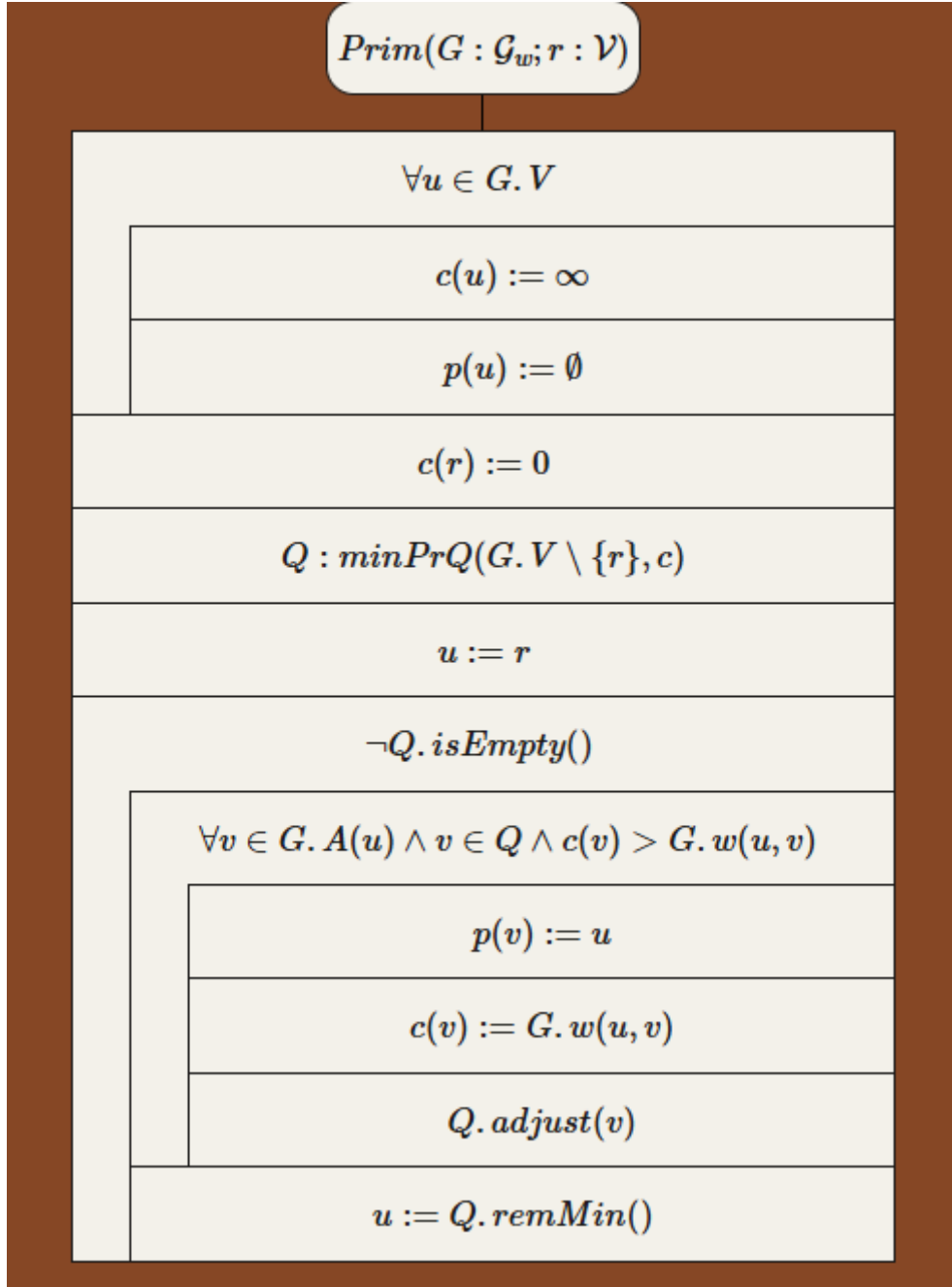
2.4. Minimális feszítőfák

2.4.1. Kruskal algoritmus



$$O(m * \log n).$$

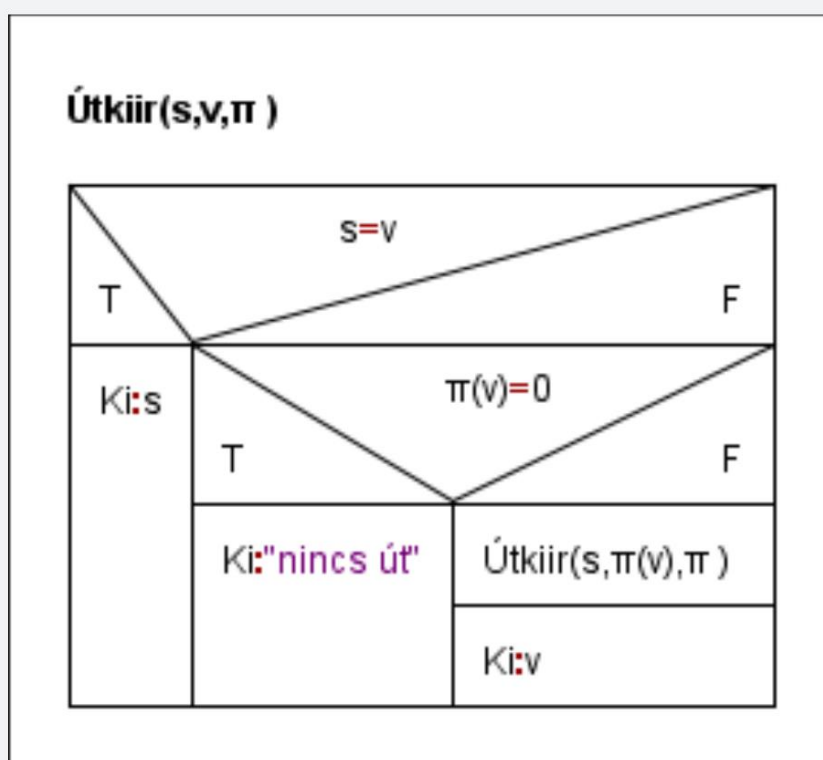
2.4.2. Prim algoritmus



$$MT_{Prim}(n, m) \in O(m * \log n).$$

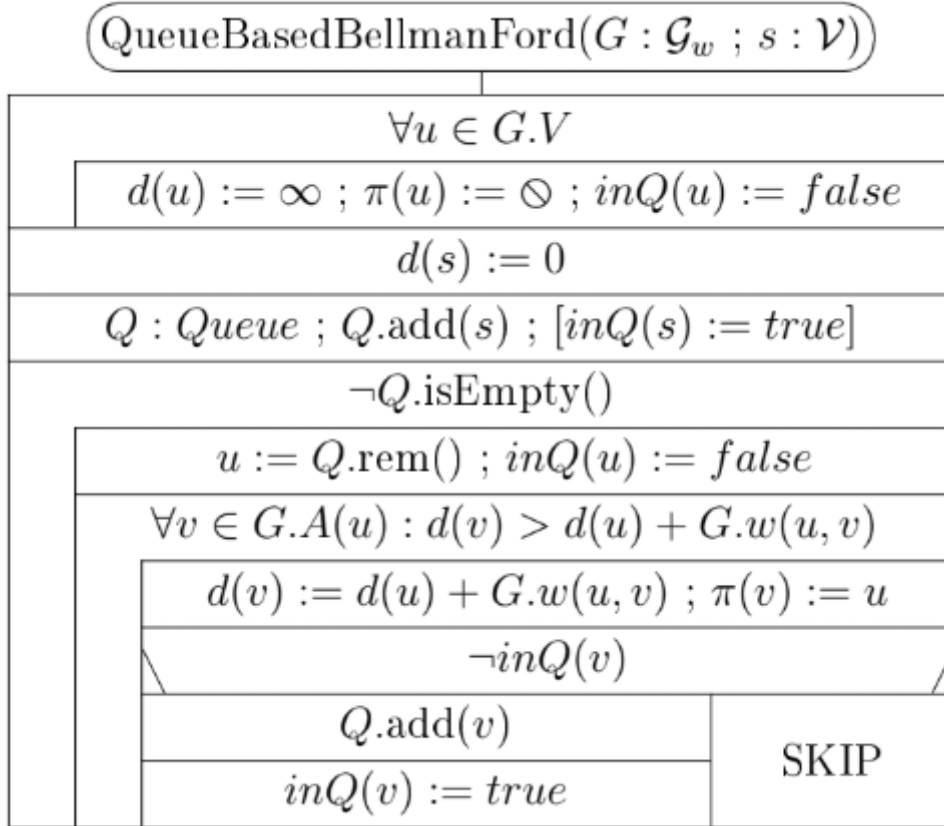
2.5. Legrövidebb utak egy forrásból

1. A Dijkstra algoritmus eredményeképpen megkaptuk a $d[1..n]$ és a $\pi[1..n]$ tömböket, amik a gráf csúcsainak d és π értékeit tartalmazzák. Írjuk meg az **útkiíró**(d, π, v) rekurzív függvényt, ami a $d[v]$ értékkel tér vissza, és kiírja a start csúcsból a v csúcsba vezető optimális utat a következő formátumban: Az úton tetszőleges x csúcs „ $x:d$ ” alakban jelenik meg, ahol d az x csúcs d -értéke. Minden él „ $--w-->$ ” alakban jelenik meg, ahol w az él súlya. Adott ehhez a kiír(y) eljárás, ami tetszőleges y paramétert kiír. Ha például a gráfnak három csúcsa van, $d[1..3] = \langle 4; 9; 0 \rangle$ és $\pi[1..3] = \langle 3; 1; 0 \rangle$, akkor a startcsúcsból a 2 indexű csúcsba vezető optimális út a következő alakban jelenjen meg:
3:0--4-->1:4--5-->2:9



2.5.2. Sor-alapú Bellman-Ford algoritmus

1.a, Mit számol ki a *Sor-alapú (Queue-based) Bellman-Ford* algoritmus? Adja meg a struktogramját! **1.b,** Mit értünk a fenti program futásának *menetei*



2.5.3. DAG lerövidebb utak egy forrásból

1.a, Írja le röviden, szóban vagy struktogrammal, azt a tanult algoritmust, mellyel irányított, súlyozott, *körmentes* gráfokra a leghatékonyabb módon határozhatjuk meg a start csúcsból a többi csúcsba vezető legrövidebb utak fáját! (Negatív élköltiségek is megengedettek.) **1.b,** Mekkora a műveletigénye? Miért? **1.c,** Szemléltesse a működését az alábbi gráfon, ⁸

Működése:

Elkészítjük a gráf egy **topologikus rendezését** mélységi keresés (DFS) segítségével. A topologikus rendezés csak az s -ből elérhető részgráfot próbálja topologikusan rendezni. A topológikus rendezést érdemes úgy elkészíteni, hogy a mélységi bejárás során a befejezési idők szerinti sorrendben berakjuk a csúcsokat egy verembe. Ha a gráf mégis tartalmazna kört, a DFS futása közben ez kiderülne, és akkor leáll az algoritmust. Ezután **kiterjesszük a csúcsokat a topologikus rendezésnek megfelelő sorrendben**, így végzünk fokozatos közelítést. Azaz mindig kivesszünk egy csúcsot a veremből és erre végzünk fokozatos közelítést.

Műveletigény:

Minden csúcsot és élt legfeljebb egyszer terjesztünk ki. Ezek alapján a futási idő a topologikus rendezéssel és a csúcsok kiterjesztésével együtt:

- $n = |G, V|$ (csúcsok száma), $m = |G, E|$ (élek száma)
- $MT(n, m) \in \Theta(n + m)$
- $mT(n, m) \in \Theta(n)$

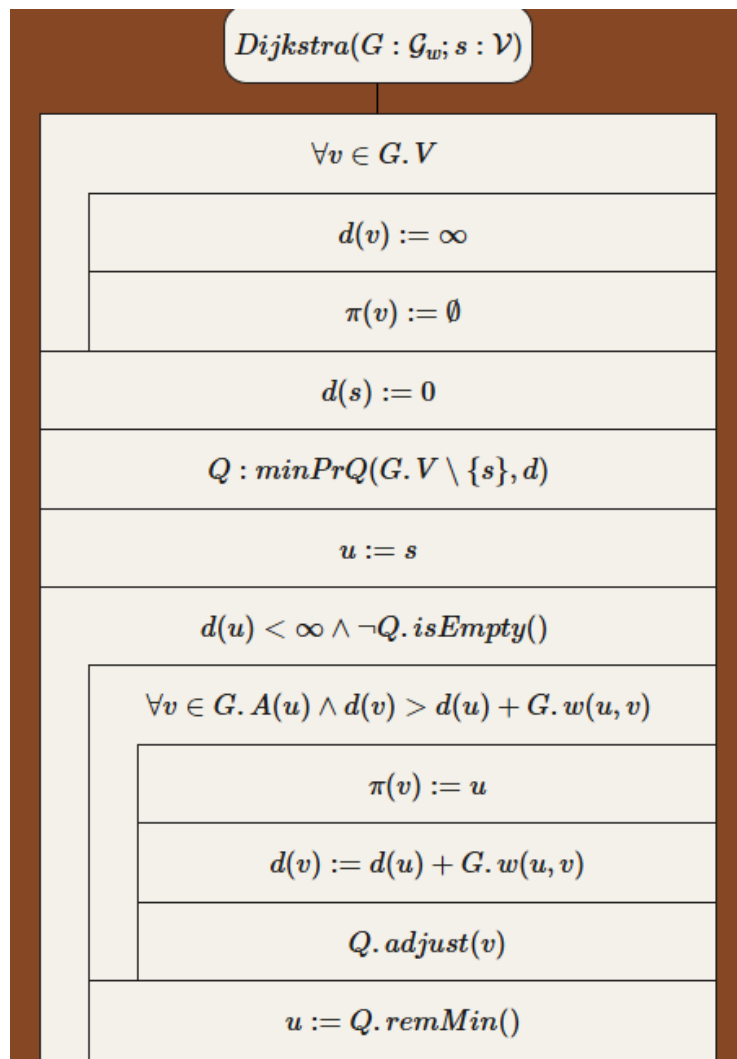
2.5.4. Dijkstra algoritmusa

1.a, Mit számol ki a *Dijkstra* algoritmus? Mekkora a műveletigénye n csúcsú gráf esetén, ha a prioritásos sort rendezetlen tömbbel reprezentáljuk? Miért?

1.b, Szemléltesse a működését az alábbi irányítatlan gráfon az **a** csúcsból indítva, az előadásról ismert módon!⁹. Rajzolja le a legrövidebb utak fáját is, ami az eredményből adódik!

a – b, 2; c, 1; d, 4. b – d, 0. c – d, 2; e, 2. d – e, 1. e.

2.a, Mit számol ki a Dijkstra algoritmus? Adja meg a struktogramját! **2.b,**



Ha a prioritásos sort minimum kupaccal valósítjuk meg:

- $n = |G.V|$ (csúcsok száma), $m = |G.E|$ (élek száma)
- $MT(n, m) \in O((n + m) \cdot \log n)$
- $mT(n, m) \in \Theta(n)$

Egyéb műveletigényei:

Ha a prioritásos sort tömbbel valósítjuk meg:

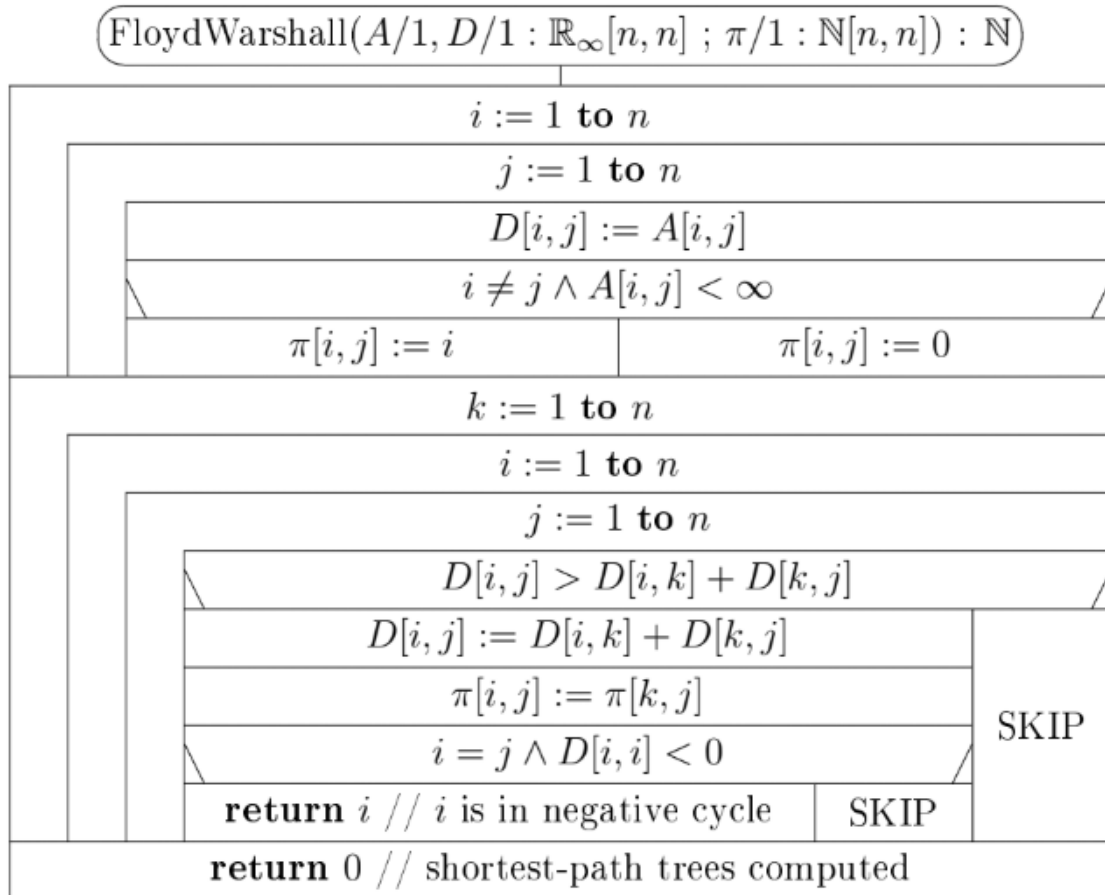
- $n = |G.V|$ (csúcsok száma), $m = |G.E|$ (élek száma)
- $MT(n, m) \in O(n^2)$, mivel az egész tömböt végig kell nézni

Ha a prioritásos sort Fibonacci kupaccal valósítjuk meg:

- $n = |G.V|$ (csúcsok száma), $m = |G.E|$ (élek száma)
- $MT(n, m) \in O(n \cdot \log n + m)$

2.6. Legrövidebb utak minden csúcspárra

Floyd-Warshall algoritmus (Floyd-Warshall algorithm)

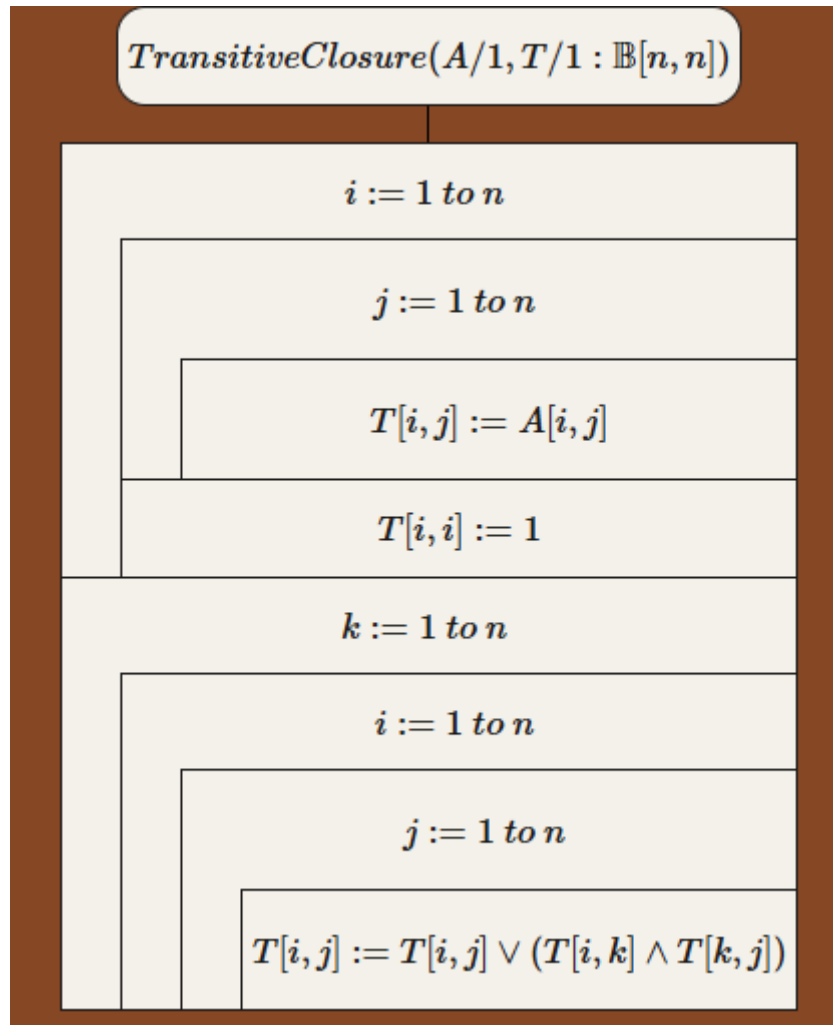


Műveletigény:

- $MT(n) \in \Theta(n^3)$
- $mT(n) \in \Theta(n^2)$

2.7. Irányított gráf tranzitív lezártja

1.a, Mit jelent egy gráf tranzitív lezártja, amit *Warshall* algoritmus számol ki? **1.b,** Tegyük fel, hogy a gráfnak n csúcsa van! Mi a $\langle T^{(k)} : k \in 0..n \rangle$ mátrix sorozat definíciója? Mi a rekurzív képlete? **1.c,** Adja meg az algoritmus struktogramját! Mekkora a műveletigénye n csúcsú gráf esetén? Miért



Műveletigény:

A Floyd–Warshall-algoritmussal megegyezően a futási idő $\Theta(n^3)$ lesz. A gyakorlatban gyorsabban hajtódnak végre a logikai műveletek, mint az egész számokon végzett aritmetikai műveletek.